

Analyse des processus - Exercice systèmes d'exploitation

Introduction

Cet exercice analyse 5 programmes en C utilisant l'appel système `fork()`. Pour chaque programme, on détermine :

- Le nombre total de processus créés
- L'arbre généalogique des processus
- Les valeurs des variables pour chaque processus

Concept	Description
<code>fork()</code>	Crée un nouveau processus identique au parent
Valeur de retour	Parent reçoit le PID de l'enfant, enfant reçoit 0
Arbre généalogique	Représentation hiérarchique des relations parent-enfant

Code 1

```
1 int main() {  
2     fork();  
3     fork();  
4     fork();  
5 }
```

Analyse

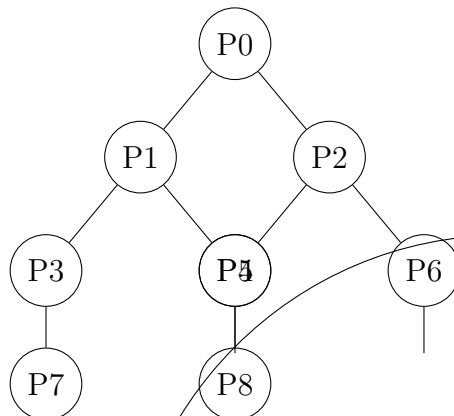
Trois appels successifs à `fork()` sans condition.

Nombre de processus : 8

- 1er `fork()` : 2 processus
- 2ème `fork()` : $2 \times 2 = 4$ processus
- 3ème `fork()` : $4 \times 2 = 8$ processus

Formule : 2^n où $n = 3$ (nombre de `fork()`) $\rightarrow 2^3 = 8$

Arbre généalogique



Étapes :

1. $P0 \rightarrow P1$ (fork 1)
2. $P0 \rightarrow P2$, $P1 \rightarrow P3$ (fork 2)
3. $P0 \rightarrow P4$, $P1 \rightarrow P5$, $P2 \rightarrow P6$, $P3 \rightarrow P7$ (fork 3)

Total : 8 processus (P0 à P7)

Valeurs des variables

Aucune variable dans ce code.

Code 2

```

1 int main() {
2     int x, y, z;
3     x = fork();
4     y = fork();
5     z = fork();
6 }

```

Analyse

Même structure que Code 1 mais avec stockage des valeurs de retour.

Nombre de processus : 8

Même raisonnement : $2^3 = 8$ processus.

Arbre généalogique

Même arbre que Code 1.

Valeurs des variables

Tableau des valeurs (PID hypothétiques : 1-7) :

Processus	Parent	x	y	z
P0 (initial)	-	PID1	PID2	PID3
P1 (enfant 1er fork)	P0	0	PID4	PID5
P2 (enfant 2e fork de P0)	P0	PID1	0	PID6
P3 (enfant 2e fork de P1)	P1	0	0	PID7
P4 (enfant 3e fork de P0)	P0	PID1	PID2	0
P5 (enfant 3e fork de P1)	P1	0	PID4	0
P6 (enfant 3e fork de P2)	P2	PID1	0	0
P7 (enfant 3e fork de P3)	P3	0	0	0

Règle :

- Si le processus est créé par le `fork()` qui définit la variable, cette variable = 0
- Sinon, la variable = PID de l'enfant créé par ce fork

Code 3

```
1 int main() {  
2     int x = 0;  
3     int y = 0;  
4     if (fork() > 0)  
5         x = 2;  
6     else  
7         y = 4;  
8 }
```

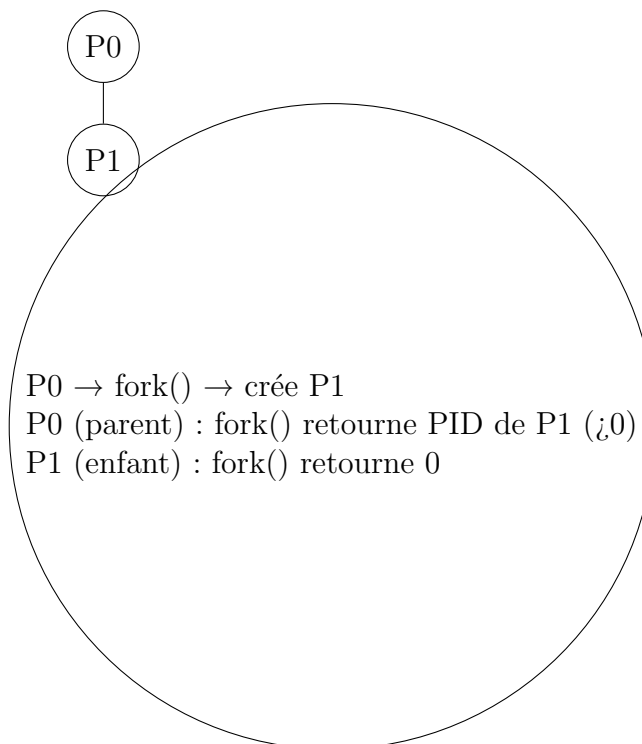
Analyse

Un seul appel à `fork()` avec condition.

Nombre de processus : 2

- Parent : `fork() != 0` → exécute `x = 2`
- Enfant : `fork() = 0` → exécute `y = 4`

Arbre généalogique



Valeurs des variables

Processus	x	y
P0 (parent)	2	0
P1 (enfant)	0	4

Code 4

```
1 int main() {  
2     if (fork() > 0)  
3         fork();  
4 }
```

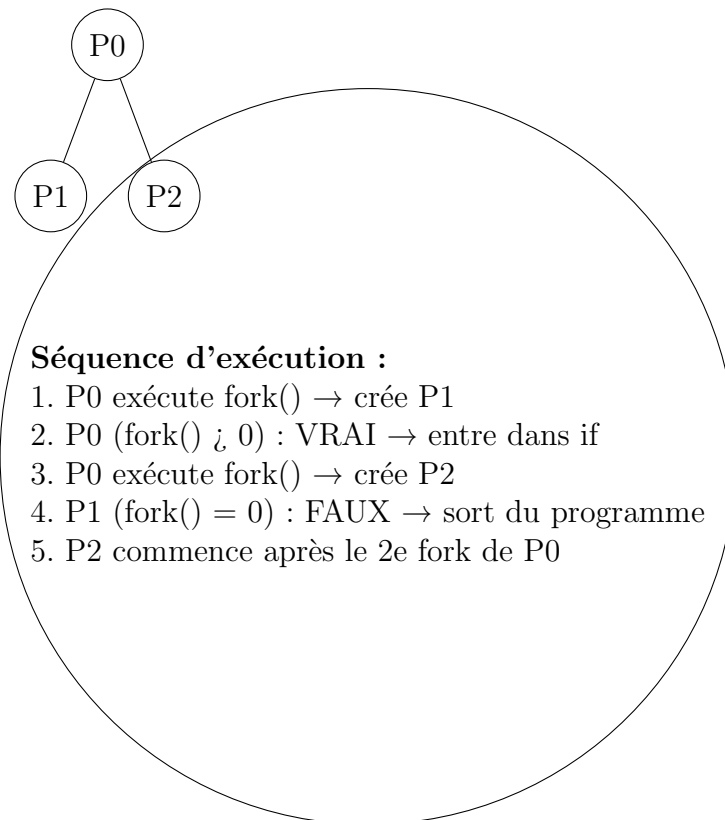
Analyse

1. Premier `fork()` : crée P1
2. Seulement le parent (P0) entre dans le if
3. P0 exécute un deuxième `fork()` : crée P2
4. P1 ne rentre pas dans le if et se termine

Nombre de processus : 3

P0, P1, P2

Arbre généalogique



Valeurs des variables

Aucune variable dans ce code.

Code 5

```
1 int main() {  
2     int cpt = 0;  
3     while (cpt < 3) {  
4         if (fork() > 0)  
5             cpt++;  
6         else  
7             cpt = 3;  
8     }  
9 }
```

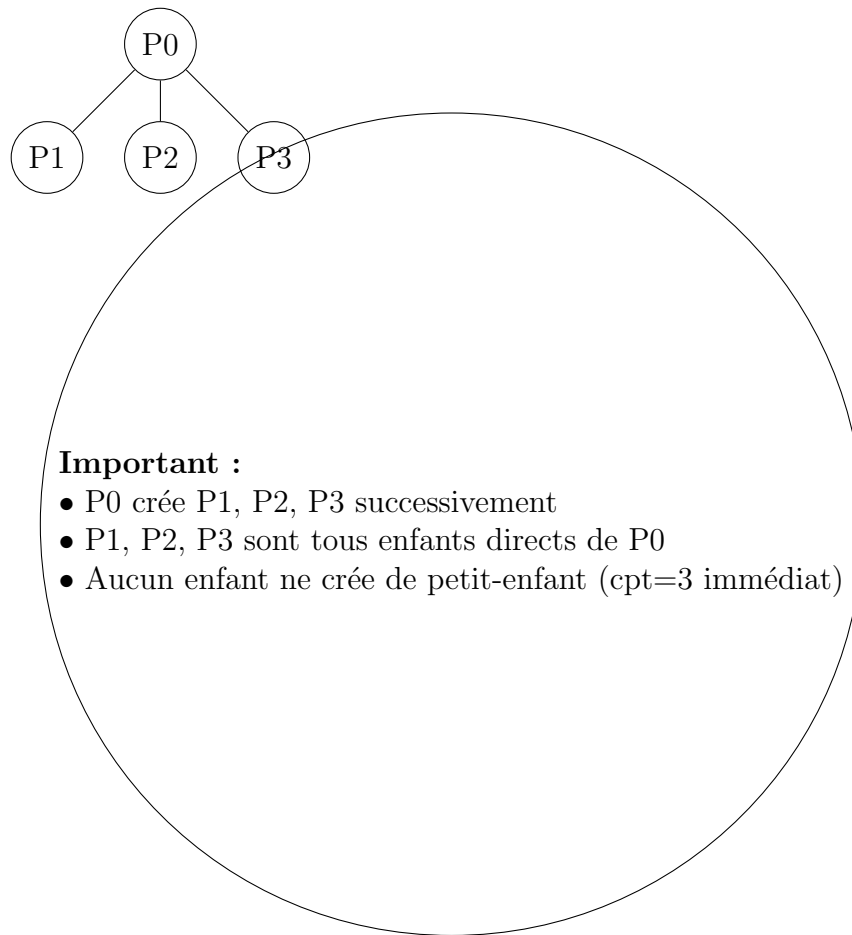
Analyse détaillée

- `cpt` initialisé à 0
- Boucle `while (cpt < 3)`
- À chaque itération :
 - Parent : `fork() > 0` → incrémente `cpt`
 - Enfant : `fork() = 0` → met `cpt = 3` et sort de la boucle

Nombre de processus : 4

Itération	Processus actif	Action	cpt après
1	P0 (cpt=0)	fork() → crée P1	P0.cpt = 1
2	P0 (cpt=1)	fork() → crée P2	P0.cpt = 2
3	P0 (cpt=2)	fork() → crée P3	P0.cpt = 3
-	P1, P2, P3	cpt = 3 → sort boucle	cpt = 3

Arbre généalogique



Valeurs des variables

Processus	Valeur de cpt
P0	0 → 1 → 2 → 3 (final)
P1	3 (dès création)
P2	3 (dès création)
P3	3 (dès création)

Récapitulatif général

Code	Nb processus	Formule	Arbre	Valeurs des
1	8	2^3	Arbre binaire complet	Aucune
2	8	2^3	Arbre binaire complet	x,y,z = 0 ou 1 tion
3	2	2^1	Parent $\rightarrow$ Enfant	Parent : x=2 x=0,y=4
4	3	Conditionnelle	P0 $\rightarrow$ P1 et P0 $\rightarrow$ P2	Aucune
5	4	Boucle while	P0 $\rightarrow$ P1,P2,P3 (enfants directs)	P0 : cpt=3 ; P

Conclusions importantes

1. **fork() sans condition** : Chaque fork double le nombre de processus
2. **fork() avec condition** : Seul le parent (fork() $\neq$ 0) exécute le code dans le if
3. **fork() dans une boucle** : Attention aux processus qui peuvent continuer à boucler
4. **Variables** : Chaque processus a sa propre copie des variables après le fork
5. **Valeur de retour** :
 - Dans le parent : PID de l'enfant ($\neq$ 0)
 - Dans l'enfant : 0
 - En cas d'erreur : -1

Conseils pour l'analyse

1. Compter systématiquement le nombre de fork()
2. Vérifier les conditions (if, while)
3. Suivre l'exécution pas à pas pour chaque processus
4. Dessiner l'arbre au fur et à mesure de l'analyse
5. Distinguer parent et enfant par la valeur de retour de fork()